

Does Morphological Decomposition Still Help LLMs Understand Hebrew?

Or Zahavi (ID 213450703), Alon Kremerman (ID 209363829), Rafael Yzgeav (ID 207015660)

Submitted as final project report for the NLP course, RUNI, 2026

Code and full results: <https://github.com/AdonZahavi/NLP-Final->

1. Introduction

Hebrew is a morphologically rich language in which a single written word frequently bundles several grammatical units: conjunctions, prepositions, the definite article, a content stem, and pronominal suffixes. The word *וּכְשֶׁהָלַכְתִּי* (“and when I walked”) packs an entire clause into one token: *ו* (and) + *כְּשֶׁ* (when) + *הָלַכְתִּי* (I walked). For decades, Hebrew NLP pipelines treated morphological segmentation as a mandatory preprocessing step: analyzers such as YAP decompose each surface word into its constituent morphemes before any downstream task is attempted.

Modern large language models challenge this convention. They consume text through subword tokenizers (BPE and variants) trained on raw, unsegmented corpora, and they appear to handle Hebrew reasonably well without explicit morphological preprocessing. This project revisits the classic question in the LLM era:

RQ1: Does explicit morphological decomposition still improve LLM performance on Hebrew downstream tasks, or do modern models already handle Hebrew morphology internally?

RQ2 (novel): Can the hypothesized benefit of segmentation be obtained through prompting alone — by instructing the model to “mentally decompose” Hebrew words — without any preprocessing pipeline?

We answer these questions with a systematic 4×3×3 experiment: four LLMs (GPT-4o, Claude, LLaMA 3.2 3B Instruct, Mistral 7B Instruct) evaluated on three Hebrew tasks (sentiment analysis, natural language inference, and extractive question answering) under three input conditions (raw text, YAP-segmented text, and prompt-guided morphology), totaling roughly 22,600 model predictions. Beyond aggregate metrics, we contribute paired statistical analysis (exact McNemar tests), a flip analysis linking answer changes to morphological properties of the input, a tokenizer fertility analysis that explains the results mechanistically, and — a methodological contribution in its own right — the identification and correction of two evaluation artifacts that initially exaggerated the harm of segmentation: a span-spacing artifact in extractive QA, and a thinking-budget artifact in which a reasoning model returned empty answers on unfamiliar input.

Our answer to RQ1 is no: neither intervention ever significantly outperforms raw Hebrew, for any model on any task. Segmentation significantly degrades NLI for three of four models (all four directionally) and QA under strict exact match for all four (for two of four after artifact correction). Our answer to RQ2 is that there is no benefit to substitute: the morphology instruction never significantly helps any model, and it significantly harms models across the capability range — GPT-4o and Mistral on NLI, and LLaMA most severely (NLI and QA) — showing that prompt-based linguistic scaffolding does not transfer reliably.

2. Related Work

Morphological processing of Hebrew. Hebrew's clitic prefixes (*כְּשֶׁ*, *וּ*, *בְּ*, *לְ*, *הָ*, *שֶׁ*, *וּ*), pronominal suffixes, and construct state (*smichut*) motivated a long line of segmentation-first pipelines. YAP (More & Tsarfaty, 2016) performs joint morphological segmentation and parsing and remains a standard open-source analyzer for Modern Hebrew; we use it as our segmenter.

Hebrew pre-trained language models. AlephBERT (Seker et al., 2022) and DictaBERT (Shmidman et al., 2023) explicitly grappled with whether to tokenize Hebrew at the word, morpheme, or subword level, and reported

that morpheme-aware vocabularies help masked language models — evidence that this design question mattered in the pre-LLM era and motivating our re-examination for instruction-tuned LLMs.

Subword tokenization in morphologically rich languages. BPE (Sennrich et al., 2016) and related tokenizers are known to over-fragment morphologically rich languages, yielding high fertility (tokens per word) and degraded downstream efficiency relative to English (Rust et al., 2021). Segmentation-first preprocessing has likewise been standard in Arabic NLP, making our question relevant beyond Hebrew. Our tokenizer analysis (Section 6.4) quantifies fertility for Hebrew across all four models' tokenizers.

Prompting as linguistic scaffolding. Instruction-following models can sometimes be steered by meta-cognitive prompts such as chain-of-thought (Wei et al., 2022). To our knowledge, no prior work has tested whether prompting an LLM to attend to Hebrew morphology can substitute for explicit segmentation; our prompt-guided condition is a direct test of this idea.

3. Datasets

We use three Hebrew-native benchmarks from HuggingFace, sampling a fixed, seeded evaluation subset per task that is held identical across all models and conditions (SEED=42, subsets committed with SHA-256 checksums):

- Sentiment — omilab/hebrew_sentiment: user comments from the Facebook page of Israel's president. We use 500 test examples, stratified with a floor of 30 off-topic examples. During exploration we found the label inventory is positive/negative/off-topic — not neutral as commonly assumed — and prompts were phrased accordingly.
- NLI — HebArabNlpProject/HebNLI: Hebrew MultiNLI. We use the entire human-verified gold test set (884 pairs; entailment/contradiction/neutral).
- QA — Etelis/HeQ_v1: SQuAD-style reading comprehension over Hebrew Wikipedia and Geektime. We use 500 answerable test questions, sampled uniformly.

Label distributions: the sentiment subset is imbalanced (63.2% positive, 30.8% negative, 6.0% off-topic; majority-class baseline 0.632), while the NLI gold set is balanced (34.7/32.7/32.6%; majority baseline 0.347). Full EDA in analysis/extra_metrics.md.

Prompt-variant selection (Section 4.3) used development slices strictly disjoint from these evaluation subsets: unselected token_test rows for sentiment and the HebNLI validation split for NLI, so no prompt tuning ever touched test data.

4. Methodology

4.1 Experimental conditions

- raw — the original, unsegmented Hebrew text, exactly as written.
- segmented — the same text preprocessed by YAP into space-separated morphemes, with one added prompt sentence explaining that the text is morphologically segmented.
- prompt_guided (novel) — raw text plus a natural-language instruction telling the model that Hebrew words bundle clitic prefixes and suffixes and asking it to mentally decompose each word before answering.

Only the input representation and instruction prefix vary; the task template, answer format, decoding parameters, and parser are identical across conditions, so performance differences are attributable to the condition alone.

4.2 Segmentation pipeline

We compared YAP and HebPipe on a pilot sample and selected YAP (HebPipe proved unusable due to dependency rot in current transformers). YAP was built from source on Colab (Go, GOPATH mode) and run as a local server with crash-safe, cached, resumable batch processing. Validated against the omilab gold morphological segmentation, YAP achieves 30.8% exact-match and 0.875 token-level F1 on our sample; inspection shows the gap is dominated by punctuation tokenization conventions and YAP's covert definite-article normalization (e.g., *ב ה בית* → *בבית*), with a smaller class of true errors (e.g., *בהצלחה* over-split as *ב ה הצלחה*). Importantly, YAP does not merely insert spaces: it also normalizes surface forms — inserting the covert definite article (*ב ה בית* → *בבית*) and rewriting fused pronouns into analytic forms (e.g., *מן* → *ממנה* → *של היא* → *של* היא) — so the segmented condition alters the text beyond spacing, which is standard for morphological analysis but relevant to interpreting our results. Segmentation expands the text by a factor of ≈ 1.5 – 1.6 tokens per raw token depending on task.

4.3 Prompt-guided instruction selection

Five instruction variants were piloted with GPT-4o on 50-example development slices per classification task (disjoint from the evaluation subsets): a brief note (v1), a rich linguistic explanation with worked examples (v2), an example-driven variant (v3), a Hebrew-language variant (v4), and an explicit decompose-then-answer variant (v5). v2 achieved the best mean accuracy and was used for all main `prompt_guided` runs; given the small pilot (50 examples per task), differences between variants are indicative rather than significant. v5 — forcing the model to write out a decomposition before answering — scored lowest, an early hint (borne out at scale in Section 5) that imposed decomposition does not help.

Table 1: Prompt-variant pilot (GPT-4o, 50 dev examples per task).

variant	sentiment acc	NLI acc	mean
baseline (raw)	0.780	0.820	0.800
v1 brief	0.820	0.800	0.810
v2 linguistic (winner)	0.820	0.820	0.820
v3 worked example	0.800	0.760	0.780
v4 Hebrew-language	0.800	0.780	0.790
v5 decompose-first	0.720	0.760	0.740

4.4 Models and inference

Exact model versions: gpt-4o (OpenAI API), claude-sonnet-5 (Anthropic API; extended thinking disabled, Section 5.3), meta-llama/llama-3.2-3B-Instruct and mistralai/Mistral-7B-Instruct-v0.3 (HuggingFace weights, fp16 on a Colab T4 GPU); all accessed August 2026. GPT-4o and Claude represent frontier commercial models; LLaMA and Mistral represent small open models. All models are evaluated zero-shot with frozen weights and deterministic (greedy / temperature-0) decoding — no fine-tuning — so differences between conditions stem purely from input presentation. Instructions are in English with Hebrew text embedded; answers are constrained to a single English label (classification) or a copied Hebrew span (QA) and parsed strictly, with ambiguous outputs counted as errors. Parse-failure rates are $\leq 1.2\%$ in every one of the 36 `model×task×condition` cells.

4.5 Engineering for reproducibility

Every prediction is stored as a JSONL record containing the exact prompt, raw model output, parsed label, and gold answer. All API calls pass through a SQLite response cache keyed by prompt hash; runs are resumable by example id (crash-safe append-and-flush), which allowed the GPU sweeps to survive Colab preemptions

without losing or duplicating work. An automated audit validates all 36 result files for duplicates, empty outputs, degenerate predictions, prompt-condition integrity, and cross-condition id/gold consistency. One pathological 2,276-token comment exceeded T4 memory with Mistral and was truncated head+tail to 2,048 tokens (documented; single example). Total commercial API spend for all experiments — including the pilot, all three conditions, and the artifact-correction re-runs — was \$10.65 (\$7.30 Anthropic, \$3.35 OpenAI).

5. Results

Tables 2–4 present the primary metric per task (accuracy for classification, exact match for QA) under all three conditions, with deltas versus raw and exact two-sided McNemar p-values on the paired predictions. Bold/starred p-values are significant at $\alpha=0.05$.

Table 2: Sentiment accuracy (500 examples). *seg* = segmented, *pg* = prompt_guided.

model	raw	seg	Δ seg	p(seg)	pg	Δ pg	p(pg)
GPT-4o	0.734	0.730	−0.004	0.875	0.748	+0.014	0.265
Claude	0.822	0.824	+0.002	1.000	0.820	−0.002	1.000
LLaMA 3.2	0.718	0.732	+0.014	0.534	0.740	+0.022	0.300
Mistral 7B	0.752	0.744	−0.008	0.572	0.756	+0.004	0.845

Table 3: NLI accuracy (884 examples).

model	raw	seg	Δ seg	p(seg)	pg	Δ pg	p(pg)
GPT-4o	0.885	0.846	−0.038	0.0002	0.857	−0.027	0.0022
Claude	0.914	0.891	−0.023	0.0169	0.900	−0.014	0.155
LLaMA 3.2	0.459	0.431	−0.028	0.139	0.377	−0.083	<0.0001
Mistral 7B	0.514	0.464	−0.050	<0.0001	0.479	−0.035	0.0032

Table 4: QA exact match (500 examples). See Section 5.2 for the segmentation spacing artifact.

model	raw	seg	Δ seg	p(seg)	pg	Δ pg	p(pg)
GPT-4o	0.628	0.424	−0.204	<0.0001	0.648	+0.020	0.193
Claude	0.700	0.478	−0.222	<0.0001	0.682	−0.018	0.243
LLaMA 3.2	0.366	0.280	−0.086	<0.0001	0.252	−0.114	<0.0001
Mistral 7B	0.330	0.266	−0.064	0.0027	0.332	+0.002	1.000

5.1 Main findings

- Raw Hebrew is never significantly beaten. No model is significantly improved by either intervention on any task (a few non-significant positive deltas exist, e.g. GPT-4o prompt_guided QA +0.020, $p=0.19$). Segmentation significantly degrades NLI for three of four models (all four directionally) and QA under strict EM for all four; sentiment is unaffected for all models. For context, sentiment accuracies (0.72–0.82) clearly exceed the majority-class baseline of 0.632.
- Effect sizes scale inversely with model strength. For the frontier models the real NLI cost of segmentation is modest (GPT-4o −3.8, Claude −2.3 points); for the small models the prompt-guided instruction is the most harmful condition (LLaMA: −8.3 NLI, −11.4 QA, both $p<0.0001$).
- The novel condition never helps and harms across the capability range. The morphology instruction significantly degrades NLI for GPT-4o (−0.027, $p=0.0022$) and Mistral (−0.035, $p=0.0032$), and degrades

LLaMA most severely (NLI -0.083 and QA -0.114 , both $p < 0.0001$); Claude and all sentiment results are statistically unaffected. The instruction was selected in a GPT-4o pilot, yet its largest harm appears elsewhere — prompt-based linguistic scaffolding is model-dependent and does not transfer.

- A cautionary non-result: small-model sentiment. LLaMA’s accuracy rises slightly under both interventions (+1.4 segmented, +2.2 prompt-guided, both non-significant), but its macro-F1 under prompt_guided FALLS ($0.559 \rightarrow 0.522$) on this imbalanced task — consistent with drift toward the majority class rather than genuine improvement. We therefore do not claim any positive effect.

5.2 Evaluation artifact I: QA span spacing

A large part of the apparent QA collapse under segmentation is an evaluation artifact, not comprehension loss. In the segmented condition, models copy answer spans from segmented context (e.g., ב ירושלים), which fail exact-match against raw-form gold answers (בירושלים). Rescoring with space-insensitive exact match, GPT-4o’s segmented EM rises from 0.424 to 0.600 (vs raw 0.628) and Claude’s from 0.478 to 0.654 (vs 0.700). Re-running the McNemar tests on the corrected metric, the residual degradation is significant for only two of four models (Claude -0.046 , $p=0.013$; LLaMA -0.074 , $p=0.0001$), not significant for GPT-4o (-0.028 , $p=0.14$), and Mistral’s corrected segmented EM actually edges past raw (0.332 vs 0.330, $p=1.0$). Token-level F1 shows the same pattern (e.g., GPT-4o 0.723 raw $\rightarrow$ 0.617 segmented $\rightarrow$ 0.736 prompt-guided; full table in analysis/extra_metrics.md). LLaMA shows almost no spacing gap (0.280 $\rightarrow$ 0.292), indicating it was not faithfully copying segmented spans in the first place.

Table 5: QA exact match with and without space-insensitive normalization.

model	raw EM	seg EM	seg EM (space-insensitive)
GPT-4o	0.628	0.424	0.600
Claude	0.700	0.478	0.654
LLaMA 3.2	0.366	0.280	0.292
Mistral 7B	0.330	0.266	0.332

5.3 Evaluation artifact II: thinking-budget starvation

Our first full run produced a dramatic headline — Claude losing 18.8 NLI points under segmentation — that turned out to be almost entirely an artifact. Claude is a reasoning model that emits internal “thinking” tokens before its visible answer; under our tight answer budgets (8 tokens for classification), thinking could consume the entire budget and yield an empty response, which our strict scorer counted as an error. Crucially, this happened far more often on manipulated input: on NLI, 5.3% of raw inputs produced empty responses versus 26.2% of segmented inputs (QA: 18.0% vs 27.2%). After disabling extended thinking and re-running the affected examples, Claude’s segmented NLI “collapse” shrank from -18.8 to -2.3 points. We draw two lessons. Methodologically: when evaluating reasoning models with constrained output budgets, empty responses must be detected and handled, or they silently masquerade as task errors. Scientifically: the empty-response rate is itself a signal — manipulated input produced roughly five times more budget-exhausted (empty) responses, consistent with increased processing effort even where final accuracy was ultimately unaffected (we did not measure thinking length directly). Pre-correction counts are documented in analysis/empty_rates.md; the pre-correction result files are preserved in the repository’s git history.

6. Analysis

6.1 Flip analysis

For each model×task we identify examples whose correctness flipped between raw and each condition. Flips are predominantly harmful: pooled across models, the segmented condition produces 320 harmful vs 197 helpful flips on NLI and 392 vs 104 on QA. For Claude on NLI the corrected ratio is 42 harmful vs 22 helpful under segmentation — a real but moderate asymmetry, in sharp contrast to the artifact-inflated pre-correction figures.

6.2 Are flips driven by morphological complexity?

We computed morphological features of every example from its YAP output (expansion ratio; clitic-prefix density). On sentiment, harm concentrates in heavily-segmented examples: harmful flips average expansion 1.70 vs 1.51 for stable examples, while helpful flips average 1.43. On NLI and QA, however, flipped and stable examples are morphologically indistinguishable (e.g., NLI: 1.60 vs 1.59). The degradation there is therefore not concentrated where morphology is complex — it behaves like a uniform distribution shift: models simply read Hebrew worse when its spacing no longer matches the distribution they were trained on.

6.3 Qualitative examples

Native-speaker inspection of harmful Claude×NLI flips (annotated in analysis/flip_annotations.md) reveals a four-way taxonomy of how segmentation causes errors:

- Meaning-changing normalization. In nli-163, YAP rendered לעולם ("ever", idiomatic under negation: "one must never ignore reality") as ל ה עולם — literally "to the world" — erasing the temporal idiom. The model answered a hypothesis that no longer says what its author wrote.
- Segmenter error destroying a content word. In nli-178, מקורם ("their origin", stem מקור + possessive ם) was mis-split as מ קורם, treating the initial מ as the preposition "from" and leaving a non-word — the concept carrying the entire question vanished.
- Named-entity damage. In nli-229, the actor's name וינס ווהן (Vince Vaughn) had its initial ו stripped as the conjunction "and", yielding וינס ווהן — segmenters cannot distinguish a name-initial vav from a clitic.
- Harm without any error. In nli-138, every split is a genuine clitic boundary (ו ניגב, ה ספרים) and no meaning changed — yet the model still flipped from a correct entailment to neutral. This control case directly supports the distribution-shift interpretation of Section 6.2: even error-free segmentation degrades comprehension.

6.4 Tokenizer analysis: the mechanism

Why doesn't explicit segmentation help? Because subword tokenizers already segment. Measured on our sentiment texts, every tokenizer fragments raw Hebrew far beyond English (Table 6): GPT-4o's tokenizer needs 1.79 tokens per Hebrew word (vs 1.06 for English), Claude's 3.58, and the small open models' ≈4.4–4.6 — approaching character level. The models therefore never see whole Hebrew words in any condition; explicit morpheme boundaries add little information the tokenizer hasn't already imposed, while shifting the input away from the training distribution. Tokenizer quality also tracks baseline capability: the model ranking by Hebrew fertility (GPT-4o best, LLaMA/Mistral worst) mirrors the performance ranking. Finally, segmentation lengthens sequences (2.12–5.15 tokens per raw word vs 1.79–4.59), i.e., roughly 6–18% more compute and API tokens depending on model (GPT-4o +18%, Claude +14%, Mistral +12%, LLaMA +6%) to obtain no accuracy benefit.

Table 6: Tokenizer fertility (tokens per whitespace word), 300 sentiment examples.

tokenizer	Hebrew raw	Hebrew segmented	seg cost (per raw word)	English
-----------	------------	------------------	-------------------------	---------

GPT-4o (o200k)	1.79	1.44	2.12	1.06
Claude (API sample)	3.58	2.94	4.09	1.76
LLaMA 3.2	4.37	3.17	4.65	1.06
Mistral 7B	4.59	3.51	5.15	1.09

7. Discussion and Limitations

Taken together, the analyses tell one coherent story. Thirty years of Hebrew NLP practice assumed models need morphological decomposition; instruction-tuned LLMs do not. Their subword tokenizers perform implicit segmentation, their training distribution is raw text, and both of our interventions — changing the input to match linguistic theory, or asking the model to think about morphology — move the input or the computation away from what the model knows, with null-to-negative results whose magnitude scales inversely with model capability. Even where accuracy survives, manipulated input exacts a hidden cost: Claude produced far more budget-exhausted responses (Section 5.3) and every model consumed 6–18% more tokens on segmented input.

We also emphasize the methodological arc: our two most dramatic initial findings — a 20-point QA collapse and Claude’s apparent morphology fragility — were both largely evaluation artifacts, detected through qualitative inspection of per-example outputs and corrected via re-scoring and re-running. The corrected effects are smaller but far more trustworthy, and both artifact mechanisms (span-form mismatch; thinking-budget starvation) are reusable warnings for anyone evaluating LLMs on preprocessed input.

Limitations. (1) Each condition uses a single prompt formulation; prompt sensitivity is mitigated by the pilot but not eliminated. (2) The winning instruction was selected using GPT-4o only, which we later showed matters — per-model pilots might yield less harmful instructions. (3) YAP’s segmentation is imperfect (≈ 0.875 token F1 vs gold); some segmented-condition harm may reflect segmenter errors rather than segmentation per se, though the uniformity of the degradation argues against this as the main driver. (4) QA conclusions depend on span normalization; we report both scorings. (5) Single run per cell with deterministic decoding; McNemar tests address item sampling, not decoding variance. We report uncorrected p-values across 24 tests; under a Bonferroni correction ($\alpha \approx 0.002$) the strongest results survive but Claude’s segmented-NLI effect ($p = 0.017$) would not, softening “three of four” to “two of four” at that threshold. (6) Claude was evaluated with extended thinking disabled; results may differ under other reasoning configurations.

8. Conclusion

We asked whether morphological decomposition still helps LLMs understand Hebrew. Across four models, three tasks, three input conditions and $\sim 22,600$ predictions, the answer is no: no intervention ever significantly outperforms raw Hebrew. Explicit YAP segmentation yields significant but modest degradation for frontier models and larger harm for small ones; the novel prompt-guided morphology condition provides no benefit anywhere and significantly harms models across the capability range — most severely the small model it was hypothesized to help most. Mechanistically, subword tokenizers already impose fine-grained segmentation on Hebrew, making explicit morphology redundant, distribution-shifting, and 6–18% more expensive. For practitioners, the recommendation is simple: feed LLMs raw Hebrew. For the field, the result marks a genuine shift — preprocessing wisdom from the statistical-NLP era does not transfer to instruction-tuned LLMs, and “helping” them with linguistic structure can actively hurt, subtly distort evaluation, and increase processing effort.

References

- More, A. and Tsarfaty, R. (2016). Data-Driven Morphological Analysis and Disambiguation for Morphologically Rich Languages and Universal Dependencies. Proceedings of COLING 2016.
- Seker, A., Bandel, E., Bareket, D., Brusilovsky, I., Greenfeld, R., and Tsarfaty, R. (2022). AlephBERT: Language Model Pre-training and Evaluation from Sub-Word to Sentence Level. Proceedings of ACL 2022, pp. 46–56.
- Shmidman, S., Shmidman, A., and Koppel, M. (2023). DictaBERT: A State-of-the-Art BERT Suite for Modern Hebrew. arXiv:2308.16687.
- Amram, A., Ben-David, A., and Tsarfaty, R. (2018). Representations and Architectures in Neural Sentiment Analysis for Morphologically Rich Languages: A Case Study from Modern Hebrew. Proceedings of COLING 2018, pp. 2242–2252. (hebrew_sentiment dataset)
- Cohen, A., Merhav-Fine, H., Goldberg, Y., and Tsarfaty, R. (2023). HeQ: a Large and Diverse Hebrew Reading Comprehension Benchmark. Findings of EMNLP 2023, pp. 13693–13705. (HeQ dataset)
- Sennrich, R., Haddow, B., and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. Proceedings of ACL 2016.
- Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., and Gurevych, I. (2021). How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. Proceedings of ACL 2021.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- HebNLI (HebArabNlpProject), dataset card, HuggingFace.

Appendix A: Prompt templates

For transparency, the exact task templates (identical across conditions; the {prefix} slot carries the condition text — empty for raw, the segmentation note for segmented, the morphology instruction for prompt_guided):

- Sentiment: "{prefix}You are given a comment written in Hebrew, posted on the Facebook page of Israel's president. Classify its sentiment toward the president/post as exactly one of: positive, negative, off-topic. (off-topic means the comment does not relate to the post at all.) Comment: {text} Answer with one word only: positive, negative, or off-topic."
- NLI: "{prefix}You are given a premise and a hypothesis in Hebrew. Decide the relationship between them: entailment (the premise implies the hypothesis), contradiction (they cannot both be true), or neutral (neither). Premise: {premise} Hypothesis: {hypothesis} Answer with one word only: entailment, contradiction, or neutral."
- QA: "{prefix}Read the Hebrew paragraph and answer the question. Answer ONLY with the shortest exact span copied from the paragraph, in Hebrew. Do not add explanations. Paragraph: {context} Question: {question} Answer:"
- Segmented-condition note: "Note: the Hebrew text below has been morphologically segmented — each word is split into its component morphemes, separated by spaces."
- Prompt-guided instruction (winning variant v2): "Hebrew is a morphologically rich language: a single written word may bundle a conjunction, a preposition, the definite article, a content stem, and a pronominal suffix. For example, וְכַשְׁהָלַכְתִּי is ו (and) + כַּשְׁ (when) + הָלַכְתִּי (walked) + תִּי (I) — a whole clause in one word. Common prefixes: ו (and), שְׁ (that/which), הַ (the), לְ (to), בְּ (in), מִ (from), כְּ (as/like), כַּשְׁ (when). Suffixes mark possession or object pronouns (שְׁלוֹ = בית = ביתו). While reading the text below, mentally decompose every word into its morphemes before deciding your answer."